

5-Mavzu: Mashinali o‘qitishda NumPy va Pandas paketlarining qo‘llanilishi.

1-mashg‘ulot. NumPy va Pandas bilan ma’lumotlarni qayta ishlash.

O‘quv savollari:

1. NumPy kutubxonasi asoslari (massivlar bilan ishlash)
2. Pandas kutubxonasi asoslari (ma’lumotlarni qayta ishlash)

1-o‘quv savol: NumPy kutubxonasi asoslari (massivlar bilan ishlash)

Maqsad

Kursantlar **NumPy kutubxonasining mohiyatini**, **massivlar (ndarray)** bilan ishlash tamoyillarini, **vektorlashtirilgan hisoblash** g‘oyasini, va **amaliy misollar orqali** NumPy’ning mashinali o‘qitishdagi ahamiyatini o‘zlashtiradilar.

I. NumPy nima va nega u muhim?

Qisqacha ta’rif:

NumPy (Numerical Python) — bu Python tili uchun mo‘ljallangan **matematik hisoblash va massivlar bilan ishlash kutubxonasi**. U mashinali o‘qitish, kompyuter ko‘rish, ma’lumotlar tahlili, va sun’iy intellekt tizimlarida **asosiy hisoblash dvigateli** sifatida ishlaydi.

Oddiy Python ro‘yxatlari va NumPy farqi:

Operatsiya turi	Oddiy Python ro‘yxatlari	NumPy massivlari
10 million element yig‘indisi	Sekundlab ishlaydi	Millisekundlarda ishlaydi
Har xil ma’lumot turlari	Aralash (int, str, float)	Faqat bitta turdagi elementlar
Vektor amallari	Loop orqali	To‘g‘ridan-to‘g‘ri matematik ifoda
Xotira samaradorligi	Yuqori emas	Juda yuqori

Xulosa: NumPy — bu **vektorlashtirilgan tezkor hisoblash** vositasi.

II. NumPy kutubxonasini o‘rnatish va ulash

```
pip install numpy
```

Dasturda foydalanish:

```
import numpy as np
```

Endi `np` orqali barcha NumPy funksiyalariga murojaat qilamiz.

III. NumPy massivlari (ndarray) tushunchasi

NumPy’da **massiv (array)** — bu **bir xil turdagi elementlardan** tashkil topgan **ko‘p o‘lchovli ma’lumotlar tuzilmasi**.

1D massiv (vektor)

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
print(a)
```

Natija: [1 2 3 4 5]

2D massiv (matritsa)

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b)
```

Natija:

```
[[1 2 3]
 [4 5 6]]
```

3D massiv

```
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

IV. Massivlar xususiyatlari

Har bir NumPy massivining **atributlari** mavjud:

```
print(a.ndim)    # O'lchamlar soni
print(a.shape)   # O'lcham tuzilmasi (qatorlar, ustunlar)
print(a.size)    # Umumiy elementlar soni
print(a.dtype)   # Elementlarning ma'lumot turi
```

V. Massiv yaratishning qulay usullari

NumPy'da massivlar **qo'lda** yoki **avtomatik** tarzda yaratiladi.

Funksiya	Tavsif	Misol
<code>np.zeros()</code>	0 bilan to'ldirilgan massiv	<code>np.zeros((2,3))</code>
<code>np.ones()</code>	1 bilan to'ldirilgan massiv	<code>np.ones((3,2))</code>
<code>np.arange()</code>	Oraliq'dagi sonlar ketma-ketligi	<code>np.arange(0,10,2)</code>
<code>np.linspace()</code>	Teng oraliqda sonlar	<code>np.linspace(0,1,5)</code>
<code>np.random.rand()</code>	Tasodifiy sonlar [0,1)	<code>np.random.rand(3,4)</code>

Interaktiv topshiriq:

`np.arange()` va `np.linspace()` yordamida 0 dan 10 gacha bo'lgan 10 ta sonli massiv yarating. Qaysi biri silliq oraliqda chiqadi?

VI. Massivlarda indekslash va kesish (slicing)

NumPy massivlari **indeks orqali tez kirish** imkonini beradi:

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])    # 10
print(arr[-1])   # 50
print(arr[1:4])  # [20 30 40]
```

2D massivlar uchun:

```
mat = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(mat[1,2])    # 6
print(mat[:,1])    # 2,5,8 – 2-ustun
print(mat[0:2, 0:2])# 2x2 kichik matritsa
```

VII. Arifmetik va statistik amallar

NumPy orqali **to‘g‘ridan-to‘g‘ri matematik ifodalar** bajariladi:

```
a = np.array([1,2,3,4])
b = np.array([10,20,30,40])

print(a + b)
print(a * b)
print(a ** 2)
print(np.sqrt(b))
```

Statistik funksiyalar:

```
data = np.array([10, 20, 30, 40, 50])
print("O‘rtacha:", np.mean(data))
print("Median:", np.median(data))
print("Min:", np.min(data))
print("Max:", np.max(data))
print("Standart og‘ish:", np.std(data))
```

VIII. Tasodifiy massivlar bilan ishlash

Tasodifiy sonlar **ML modelini sinovdan o‘tkazish** uchun juda zarur.

```
rand_nums = np.random.rand(3,3)
rand_ints = np.random.randint(0,100,(4,4))
```

Interaktiv mashq:

5x5 o‘lchamli tasodifiy butun sonlardan iborat massiv yarating va quyidagilarni toping:

- Eng katta son
- Eng kichik son
- O‘rtacha qiymat

IX. Massivlarni shaklini o‘zgartirish

```
arr = np.arange(12)
print(arr.reshape(3,4)) # 3x4 matritsa
```

Birlash va ajratish

```
x = np.array([[1,2],[3,4]])
y = np.array([[5,6]])

print(np.concatenate((x, y), axis=0))
```

X. Amaliy mini loyiha: Baholarni tahlil qilish

Misol:

100 ta talabadan olingan imtihon natijalarini tahlil qilamiz.

```
import numpy as np

grades = np.random.randint(50, 100, 100)
print("Barcha baholar:", grades)
print("O'rtacha baho:", np.mean(grades))
print("Eng yuqori:", np.max(grades))
print("Eng past:", np.min(grades))
print("90 dan yuqori baholarning foizi:", np.sum(grades > 90), "%")
```

Natija: NumPy yordamida **bitta qatorli kodlar** orqali katta hajmdagi statistik tahlilni bajarish mumkin.

XI. Mashinali o'qitishdagi ahamiyati

NumPy — bu:

- **TensorFlow, PyTorch, Scikit-learn, Pandas** kabi kutubxonalarning **poydevori**;
- Ma'lumotlarni **tayyorlash, normallashtirish, matritsa amallari** uchun ishlatiladi;
- Modellar uchun **kirish (input) va chiqish (output)** ma'lumotlarini **ndarray** ko'rinishida uzatadi.

Yakuniy xulosa

- NumPy — **Python'ning matematik yuragi**.
- U **tez, ixcham, vektorlashtirilgan** hisoblashlarni amalga oshiradi.
- Mashinali o'qitishning har bir bosqichida (ma'lumot tayyorlash, tahlil, modelga kiritish) **NumPy massivlari** ishlatiladi.

Mustahkamlovchi interaktiv savollar

1. `np.arange(0, 10, 2)` va `np.linspace(0, 10, 5)` farqini izohlang.
2. `3x3` o'lchamli massiv yarating va diagonal elementlarini ajrating.
3. `np.mean()` va `np.median()` natijalari nima sababli farq qiladi?
4. Qanday hollarda `reshape()` funksiyasi ishlamaydi?
5. `np.random.randint()` va `np.random.rand()` o'rtasidagi farqni tushuntiring.

Uyga vazifa

1. 10×10 o'lchamli tasodifiy massiv yaratib, uni 5×20 shaklga o'zgartiring.
2. Har bir ustun bo'yicha o'rtacha qiymatni hisoblang.
3. 1D massivdagi barcha juft sonlarni ajratib oling.
4. `reshape`, `concatenate`, `split` funksiyalarini qo'llab mini dastur yozing.

2-o'quv savol: Pandas kutubxonasi asoslari (ma'lumotlarni qayta ishlash)

Maqsad

Kursantlarga **Pandas kutubxonasi** yordamida ma'lumotlarni o'qish, tozalash, tahlil qilish, filtrlash va guruhlash ko'nikmalarini o'rgatish. Shuningdek, real CSV/Excel ma'lumotlar bilan ishlash jarayonini amalda ko'rsatish.

I. Pandas nima va nega u kerak?

Qisqacha ta'rif:

Pandas — bu **Python** tilidagi ma'lumotlarni qayta ishlash va tahlil qilish uchun mo'ljallangan kuchli kutubxona. U ikki asosiy ma'lumot tuzilmasiga asoslanadi:

1. **Series** — bitta o'lchamli ma'lumotlar (ustun ko'rinishida)
2. **DataFrame** — ikki o'lchamli jadval (qatorlar va ustunlardan iborat)

Pandas nima qiladi?

Vazifa	Tavsif
Ma'lumot o'qish	CSV, Excel, SQL, JSON fayllardan o'qish
Tozalash	Bo'sh, noto'g'ri yoki takror ma'lumotlarni aniqlash va to'ldirish
Tahlil	Statistik ko'rsatkichlar, o'rtacha, median, dispersiya va h.k.
Filtrlash	Shart asosida kerakli satrlarni ajratish
Guruhlash	Ma'lumotlarni toifalarga ajratish va yig'indisini olish
Saqlash	Natijani yangi CSV yoki Excel faylga yozish

Qisqasi, **Pandas** — bu ma'lumotlar bilan ishlovchi tahlilchi uchun “ish stoli”.

II. Kutubxonani ulash va dastlabki sozlash

```
import pandas as pd
```

Versiyani tekshirish:

```
print(pd.__version__)
```

III. Pandas ma'lumot tuzilmalari

1. Series (bitta ustunli tuzilma)

```
import pandas as pd
```

```
s = pd.Series([10, 20, 30, 40], name="Baholar")
print(s)
```

Natija:

```
0    10
1    20
2    30
3    40
Name: Baholar, dtype: int64
```

Har bir elementga **indeks** biriktirilgan.

`Series` — bu **NumPy array** + **indeks** + **nom** kombinatsiyasi.

2. DataFrame (jadval shaklidagi tuzilma)

```
data = {
    "Ism": ["Ali", "Vali", "Dilnoza"],
    "Yosh": [20, 22, 19],
    "Bahosi": [88, 92, 75]
}

df = pd.DataFrame(data)
print(df)
```

Natija:

```
      Ism  Yosh  Bahosi
0     Ali    20      88
1     Vali    22      92
2  Dilnoza    19      75
```

IV. DataFrame bilan asosiy amallar

1. Jadvalni ko‘rish

```
print(df.head())  # Dastlabki 5 qator
print(df.tail())  # Oxirgi 5 qator
```

2. Jadval haqida ma’lumot

```
print(df.info())  # Ustunlar, turlar, bo‘sh qiymatlar
print(df.describe()) # Statistik tahlil
```

3. Ustun va satrlarni tanlash

```
print(df["Ism"])          # Bitta ustun
print(df[["Ism", "Bahosi"]]) # Bir nechta ustun
print(df.iloc[0])         # 1-qator (indeks bo‘yicha)
print(df.loc[1])          # 2-qator (nomi bo‘yicha)
```

V. Ma'lumotlarni o'qish va yozish

1. CSV dan o'qish:

```
df = pd.read_csv("talabalar.csv")
```

2. Excel dan o'qish:

```
df = pd.read_excel("data.xlsx", sheet_name="Sheet1")
```

3. CSV yoki Excelga yozish:

```
df.to_csv("natija.csv", index=False)  
df.to_excel("output.xlsx", index=False)
```

VI. Filtrlash va shartli tanlash

Oddiy shart

```
# Yosh > 20 bo'lganlar  
yoshlar = df[df["Yosh"] > 20]  
print(yoshlar)
```

Bir nechta shart

```
# Bahosi 80 dan katta va yoshi 21 dan kichik  
tanlangan = df[(df["Bahosi"] > 80) & (df["Yosh"] < 21)]
```

Indekslash (loc / iloc)

```
print(df.loc[0:1, ["Ism", "Bahosi"]]) # qator va ustun bo'yicha
```

VII. Yo'qolgan qiymatlar (NaN) bilan ishlash

Mashinali o'qitishda **yo'qolgan ma'lumotlar** muhim muammo hisoblanadi.

```
# Bo'sh qiymatlarni tekshirish  
print(df.isnull().sum())  
  
# Bo'sh kataklarni 0 bilan to'ldirish  
df.fillna(0, inplace=True)  
  
# Bo'sh satrlarni o'chirish  
df.dropna(inplace=True)
```

VIII. Ma'lumotlarni o'zgartirish va yangilash

```
# Yangi ustun qo'shish  
df["Kurs"] = [1, 2, 1]
```

```
# Ustunni o'chirish
df.drop("Yosh", axis=1, inplace=True)

# Ma'lumotni tahrirlash
df.loc[0, "Bahosi"] = 95
```

IX. Ma'lumotlarni guruhlash va tahlil qilish

```
grouped = df.groupby("Kurs")["Bahosi"].mean()
print(grouped)
```

Natija:

```
Kurs
1      85.0
2      92.0
Name: Bahosi, dtype: float64
```

X. Saralash va tartiblash

```
# Bahosi bo'yicha kamayish tartibida saralash
df_sorted = df.sort_values(by="Bahosi", ascending=False)
print(df_sorted)
```

XI. Amaliy mini loyiha: Talabalar ma'lumotlarini tahlil qilish

Vazifa: CSV faylda saqlangan talabalarining baholarini tahlil qilish.

```
import pandas as pd

df = pd.read_csv("talabalar.csv")

print("O'rtacha baho:", df["Bahosi"].mean())
print("Eng yuqori baho:", df["Bahosi"].max())
print("Eng past baho:", df["Bahosi"].min())

# Eng yaxshi talaba
best = df.loc[df["Bahosi"].idxmax()]
print("Eng yaxshi talaba:", best["Ism"])
```

Natija:

Pandas yordamida bir necha qator kod orqali butun jadvalni tahlil qilish, filtrlash va xulosalar chiqarish mumkin.

XII. Pandas + NumPy integratsiyasi

Ko'pincha **Pandas** va **NumPy** birgalikda ishlatiladi:

```
import numpy as np
import pandas as pd
```



```
arr = np.random.randint(0, 100, (5, 3))
df = pd.DataFrame(arr, columns=["X", "Y", "Z"])
print(df)
print("Ustunlar bo'yicha o'rtacha:", df.mean())
```

XIII. Mashinali o'qitishdagi roli

Pandas:

- Ma'lumotlarni **tayyorlash bosqichida** ishlatiladi;
- **Yo'qolgan qiymatlar, ortiqcha ustunlar, formatlash** muammolarini hal qiladi;
- **ML modellari** uchun toza, normalizatsiyalangan dataset tayyorlaydi.

Masalan: Scikit-learn modellariga kiritiladigan ma'lumotlar ko'pincha **Pandas DataFrame** ko'rinishida bo'ladi.

XIV. Interaktiv mashqlar

1. `pandas` yordamida 10 talaba haqida jadval yarating (`Ism`, `Yosh`, `Bahosi` ustunlari bilan).
 - O'rtacha, eng katta va eng kichik baholarni toping.
 - 90 dan yuqori bahoga ega talabalarni ajrating.
2. CSV fayl yarating (`df.to_csv('baholar.csv')`) va uni qayta o'qing.
3. `fillna()`, `dropna()` va `groupby()` funksiyalarini amalda sinab ko'ring.
4. Jadvalni `Bahosi` bo'yicha kamayish tartibida saralang.

Yakuniy xulosa

Asosiy tushuncha	Qisqacha izoh
Series	Bitta ustunli massiv (bir o'lchamli)
DataFrame	Ikki o'lchamli jadval – ma'lumotlar tuzilmasi
.head(), .info(), .describe()	Ma'lumotni tez ko'rish usullari
.fillna(), .dropna()	Bo'sh qiymatlarni qayta ishlash
.groupby(), .mean(), .sort_values()	Tahlil va saralash funksiyalari
Pandas + NumPy	ML jarayonining asosiy “ma'lumotlar bosqichi” vositasi

Mustahkamlovchi savollar

1. `Series` va `DataFrame` o'rtasidagi farq nima?
2. `NaN` qiymatlar bilan ishlashning 2 xil usulini yozing.
3. `groupby()` funksiyasi nima uchun ishlatiladi?
4. `Pandas` yordamida ma'lumotlarni qanday fayllarga yozish mumkin?
5. `df.iloc[]` va `df.loc[]` farqini izohlang.

Mustaqil ish uchun topshiriqlar

1. `NumPy` yordamida 10×10 o'lchamli tasodifiy sonlardan massiv yarating va uning eng katta hamda eng kichik elementlarini toping.
2. `Pandas` yordamida talabalar jadvalini yarating: ustunlar — `Ism`, `Yosh`, `Bahosi`. O'rtacha bahoni hisoblang.
3. CSV fayldan o'qilgan ma'lumotlarda **bo'sh qiymatlarni** aniqlang va ularni 0 bilan to'ldiring.

4. Har bir ustun bo'yicha statistik ma'lumotlarni (mean, min, max) chiqaring.

Uyga vazifa

1. Quyidagi ustunlarga ega bo'lgan jadval yarating:
Ism, Familiya, Yosh, Bahosi, Kurs, Fakultet
va uni `students.csv` faylga yozing.
2. CSV fayldan o'qib:
 - o har bir kurs bo'yicha o'rtacha bahoni toping;
 - o 85 dan yuqori baholarga ega talabalarni alohida jadvalda ko'rsating;
 - o eng yosh talaba haqida ma'lumot chiqaring.
3. Jadvalni Bahosi bo'yicha kamayish tartibida tartiblang.

